COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 -Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	Oct 13, 2026	**DATE SUBMITTED:**	Oct 13, 2026
NAME:	Ignacio, Guillian Carlo V. Regala, Joaquin Luiz	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 12.1 Algorithmic Strategies

1. PROCEDURE OUTPUT

Strategy	Algorithm	Analysis
Recursion	Merge Sort Algorithm	The Idea of recursion is to solve best case{s} directly then recurs with a simpler subproblem, now in **Merge Sort** we can see how it uses recursion to divide the array into smaller sub arrays until it has single elements then it will recursively conquer at the last steps to sort the elements inside the array.
Brute Force	Linear Search Algorithm	One of the brute force strategies is **Linear Search Algorithm,** where the searching algorithm searches for a certain element inside that array, but it will search through every single one no matter how big the array is. It has a O(n) time complexity which states that it will search through every single element, it's called a brute force strategy because if we try to find that certain element we are brute forcing to search throughout

		the array of words which takes time, and it will return the element once it has been found but if not, it will return false.
Back Tracking	Depth-First Search (Tree Algorithm)	In Depth-First Search, we have three types of traversals which are PreOrder, InOrder, LevelOrder, each traversal actually back tracks when it reaches the end of the path, then it will back track to a node and find it there is any where else it could go.
Greedy	Kruskal's Algorithm (MST)	**Minimal Spanning tree** is a Greedy Strategy because it uses local optimal choices in hopes of finding the global optimum, wherein it finds the smallest available edges that do not form a cycle
Divide and Conquer	Merge Sort Algorithm	Although merge sort also is uses a recursive strategy, it also uses the divide and conquer because for the first part where we divide the elements into sub arrays we are dividing them to sort them during conquering where we put them all back inside a sorted element array

Table 12-1. Algorithmic Strategies and Examples

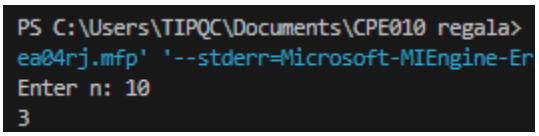
SCREENSHOT	
ANALYSIS	The memoization uses recursion to find the minimum number of steps that are needed to reach 1. It checks the possible operations of subtractive 1, dividing by 2 and dividing by 3 when it is applicable. The results are stored in an array so that the previously solved problems do not need to be calculated again. For this output of 10 the minimum steps to one is 3.

Table 12-2. Memoization Implementation

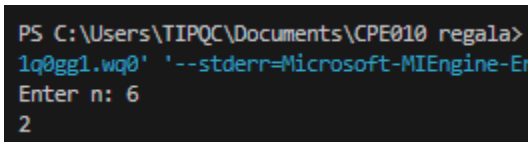
SCREENSHOT	
ANALYSIS	This bottom-up approach solves the problem by starting from the smallest value and working towards the given input. It stores the minimum number of steps for number of in the array and compares the available operations to get the lowest result. Unlike the memoization, it does not use recursive function calls because the values are calculated using a loop. For this output the 6, the program returns 2 minimum steps.

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

SUPPLEMENTARY

CODE

```cpp
#include <iostream>
#include <vector>
#include <string>
using namespace std;

const int M = 4;
const int N = 4;



int countPaths(int mat[M][N], int i, int j, int cost) {
    // Out of the matrix, or the cost is already used up
    if (i >= M || j >= N || cost < 0)
        return 0;

    // Spend this cell's cost
    cost -= mat[i][j];

    // Base case: destination cell, valid only if the cost is exactly used up
    if (i == M - 1 && j == N - 1)
        return (cost == 0) ? 1 : 0;

    // Recursive case: go right + go down
    return countPaths(mat, i, j + 1, cost) + countPaths(mat, i + 1, j, cost);
}
```

```cpp
int countPathsMemo(int mat[M][N], int i, int j, int cost,
                   vector<vector<vector<int>>> &memo) {
    if (i >= M || j >= N || cost < 0)
        return 0;

    cost -= mat[i][j];
    if (cost < 0)
        return 0;

    if (i == M - 1 && j == N - 1)
        return (cost == 0) ? 1 : 0;

    if (memo[i][j][cost] != -1)
        return memo[i][j][cost];

    memo[i][j][cost] = countPathsMemo(mat, i, j + 1, cost, memo)
                     + countPathsMemo(mat, i + 1, j, cost, memo);
    return memo[i][j][cost];
}
```

```
void printPaths(int mat[M][N], int i, int j, int cost, string path) {
    if (i >= M || j >= N || cost < 0)
        return;

    cost -= mat[i][j];
    path += "(" + to_string(i) + "," + to_string(j) + ")=" + to_string(mat[i][j]);

    if (i == M - 1 && j == N - 1) {
        if (cost == 0)
            cout << "  " << path << endl;
        return;
    }

    path += " -> ";
    printPaths(mat, i, j + 1, cost, path);
    printPaths(mat, i + 1, j, cost, path);
}
```

```
int main() {
    int mat[M][N] = {
        {4, 7, 1, 6},
        {5, 7, 3, 9},
        {3, 2, 1, 2},
        {7, 1, 6, 3}
    };
    int cost = 25;

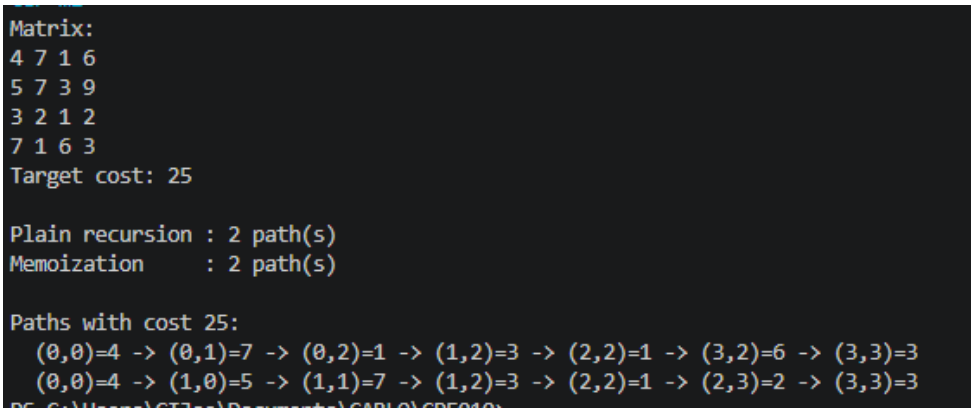
    cout << "Matrix:" << endl;
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++)
            cout << mat[i][j] << " ";
        cout << endl;
    }
    cout << "Target cost: " << cost << endl << endl;

    // Plain recursion
    cout << "recursion : " << countPaths(mat, 0, 0, cost) << " path(s)" << endl;
```

```
    // Memoization
    vector<vector<vector<int>>> memo(M, vector<vector<int>>(N, vector<int>(cost + 1, -1)));
    cout << "Memoization     : " << countPathsMemo(mat, 0, 0, cost, memo) << " path(s)" << endl;

    // Show the actual paths
    cout << endl << "Paths with cost " << cost << ":" << endl;
    printPaths(mat, 0, 0, cost, "");

    return 0;
}
```

OUTPUT	
ANALYSIS	For recursion its best case is the last cell, why? Because the remaining costs will be 0 . each cell subtracts current cell cost then will move up or down. Overspending stops a path early which causes back tracking. Memoization uses the subproblems to reduce the time.

CONCLUSION	
Ignacio	I therefore conclude that these types of algorithms are useful for different situations except for Greedy Algorithm since it just hopes that it will get global optimum. Memoization is very important because it uses caching which lessens the amount of time for the program to process and recursion is very useful when we need something to solve subproblems.
Regala	In this activity, I learned about different algorithmic strategies and how they can be used to solve problems using C++. We worked together to complete the activities and understand how recursion memoization, and bottom-up dynamic programming work. We also learned that different approaches can have different results in terms of efficiency and performance.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or

completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.